Nodebase — AI Workflow Automation Platform

Database ER Diagram & System Design (HLD / LLD)

Next.js 15 + React 19 + tRPC v11 + Prisma/Postgres + Better Auth + Inngest + BullMQ/Upstash Redis + Polar • Generated 2026

9 Postgres Tables | tRPC 3-step Procedure Chain | Inngest Execution Engine | Realtime Node Status | Encrypted Credentials | Polar Premium Gates

1. Project Overview

Nodebase is an **AI workflow automation SaaS** (n8n / Zapier style). Users build visual workflows by dropping **trigger nodes** (manual, cron, Google Form, Stripe, Telegram) and **action nodes** (HTTP request, AI providers, messaging, Google Sheets) onto a **React Flow** canvas, wiring them with edges. Workflow runs are dispatched to the **Inngest** execution engine, which topologically sorts the graph, threads a shared context through per-node executors, and streams per-node status back to the editor over realtime channels.

Frontend

- Next.js 15 App Router + Turbopack, React 19, Tailwind v4
- Visual editor: React Flow (`@xyflow/react`) + node registry
- tRPC v11 client with React Query; server prefetch → `HydrateClient` → `useSuspenseQuery`
- nuqs for URL-bound editor state; Inngest Realtime hooks for node status

Backend

- tRPC procedures: `base` → `protected` → `premium`
- Better Auth (email/password, Google, GitHub, email OTP)
- Inngest single function `executeWorkflow` + realtime channels
- BullMQ + Upstash Redis queues (email OTP, cron schedulers)
- Polar for active-subscription gating; Prisma ORM over Postgres

2. System Architecture (HLD)

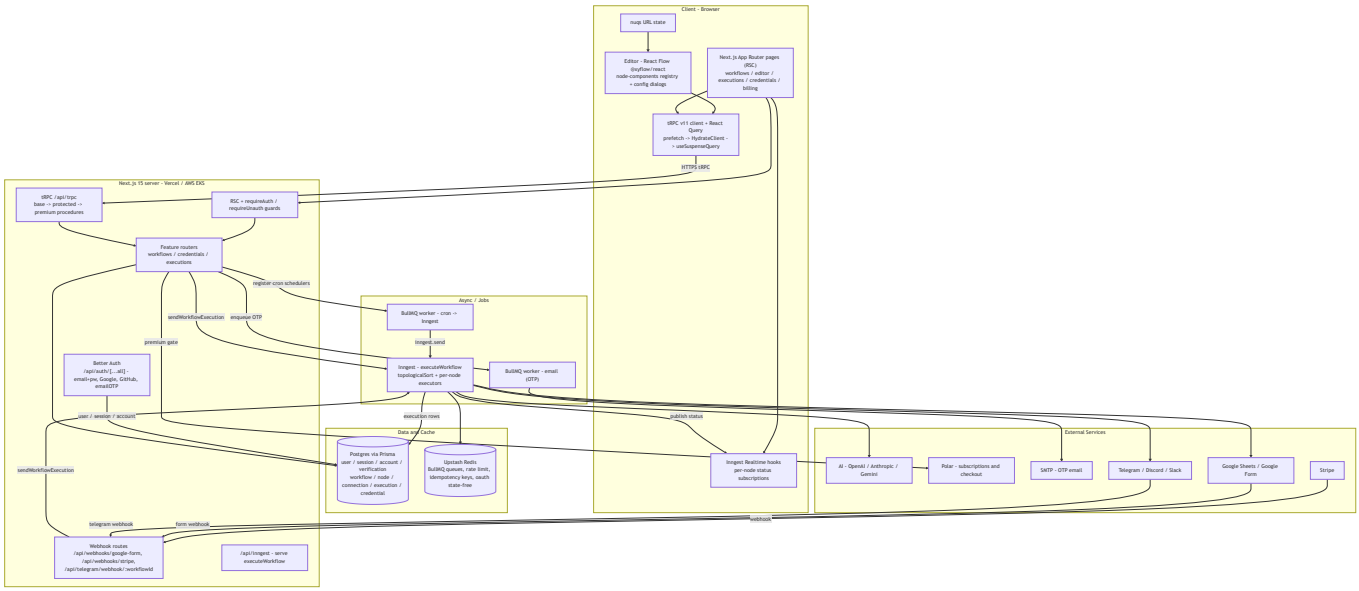


Figure 1 — High-level system architecture and component interaction

Browser clients talk to Next.js over HTTPS. tRPC (`/api/trpc`), Better Auth (`/api/auth/[...all]`) and the Inngest serve route (`/api/inngest`) are the only inbound HTTP surfaces besides the public webhook routes. Interactive requests go through the procedure chain, then feature routers (workflows / credentials / executions) run Prisma queries against Postgres. Dispatches (`sendWorkflowExecution`) land in Inngest, which executes the workflow graph, hits external providers (AI / Telegram / Discord / Slack / Sheets), and publishes status over realtime channels consumed by the editor. Cron work is registered with BullMQ schedulers that forward events back into Inngest; OTP emails are queued to a BullMQ email worker.

3. Request Lifecycle & tRPC Pipeline (LLD)

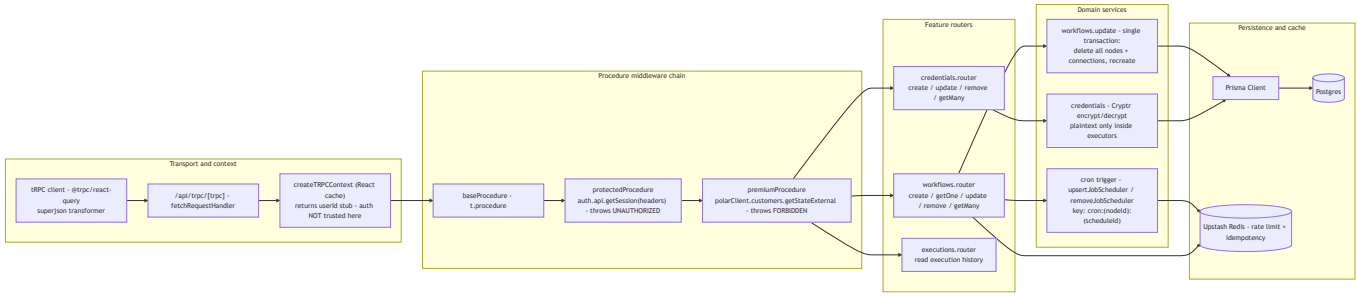


Figure 2 — Server request lifecycle: transport, procedure middleware chain, routers, domain services

1. **tRPC client** (`@trpc/react-query` + superjson) calls feature endpoints over the single `/api/trpc/[trpc]` route.
2. **createTRPCContext** is a **stub returning `userId: "user_123"`** — auth is never trusted here; it happens inside the procedures.
3. **baseProcedure** → **protectedProcedure** resolves the session via `auth.api.getSession(headers)`, throwing `UNAUTHORIZED` on missing/invalid session.
4. **premiumProcedure** (used for premium features) calls `polarClient.customers.getStateExternal` and throws `FORBIDDEN` unless an active subscription exists.
5. **Feature routers** expose `workflows`, `credentials`, `executions` mutations/reads.
6. **workflows.update** runs in **one transaction**: deletes all nodes & connections then recreates them from the editor snapshot.
7. **Credentials** are encrypted at rest with **Cryptr** (`ENCRYPTION_KEY`); plaintext is decrypted **only inside executors**, never sent to the client.
8. **Cron triggers** register/unregister BullMQ job schedulers keyed `cron:<nodeId>:<scheduleId>`; Upstash Redis backs the queue and rate-limit/idempotency keys.

4. Workflow Execution Engine (LLD)

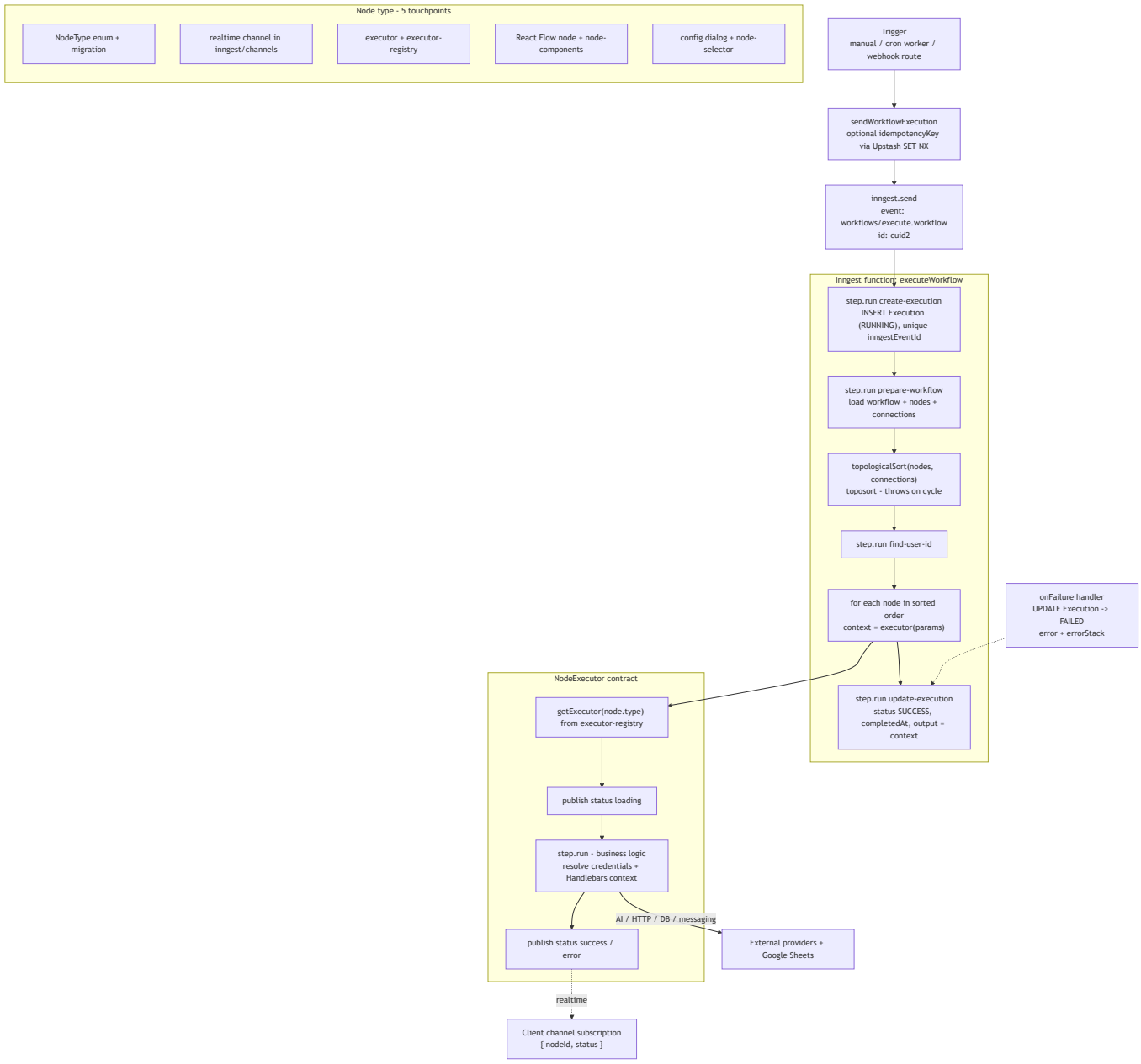


Figure 3 — Inngest `executeWorkflow`: execution lifecycle, executor contract, and node-registration touchpoints

- **Dispatch**: triggers (manual click, cron worker, webhook routes) call `sendWorkflowExecution`; an optional idempotency key uses Upstash `SET NX + EX` to dedupe webhook storms.
- **Execution record**: `step.run create-execution` inserts an `Execution` (status `RUNNING`) with a `unique inngestEventId` linking it to the Inngest run.
- **Topological sort**: nodes + connections are sorted with `toposort` (edges `fromNodeId → toNodeId`); a cycle throws before any node runs.
- **Executor contract**: each node type maps to an executor in `executor-registry`; it `publishes loading`, runs business logic via `step.run` (resolving credentials + Handlebars templating), then `publishes success/error`.
- **Context threading**: a single `WorkflowContext` record is passed node-to-node; each executor returns the merged context.
- **Realtime**: status shapes are `{ nodeId, status: "loading" | "success" | "error" }` on per-node channels consumed by the editor's React hooks.
- **Failure**: `onFailure` marks the linked `Execution` `FAILED` with `error + errorStack`; retries are `3 in prod, 0 in dev`.

5. Execution Sequence

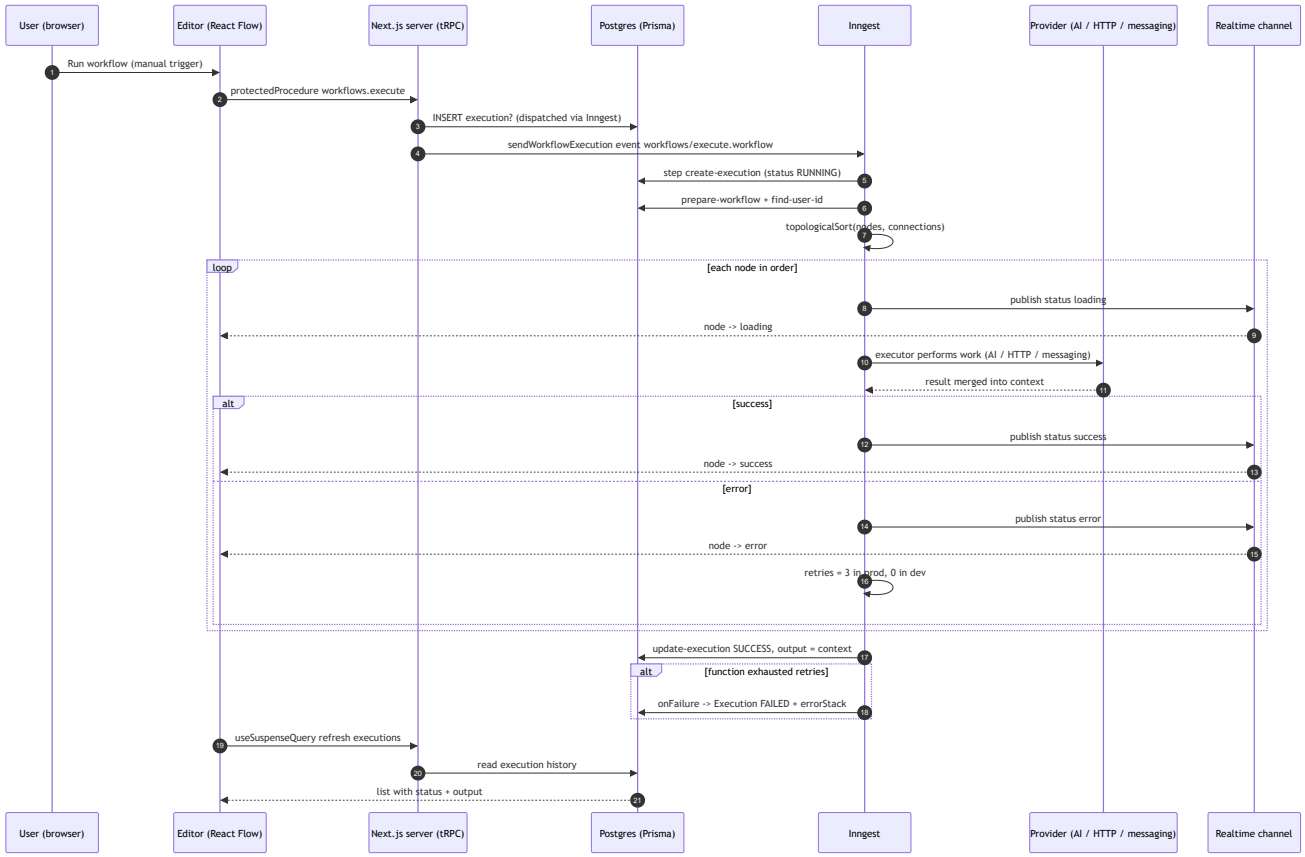


Figure 4 — End-to-end sequence for a manual workflow run, including realtime updates

The editor dispatches the run through the protected tRPC procedure; the server sends an Inngest event. The function creates the execution, sorts the graph, then walks each node — publishing loading/success/error to the client browser for live canvas feedback. On completion the execution row flips to `SUCCESS` (output = final `WorkflowContext`); exhausted retries trigger `onFailure` and set status `FAILED`.

6. Database ER Diagram

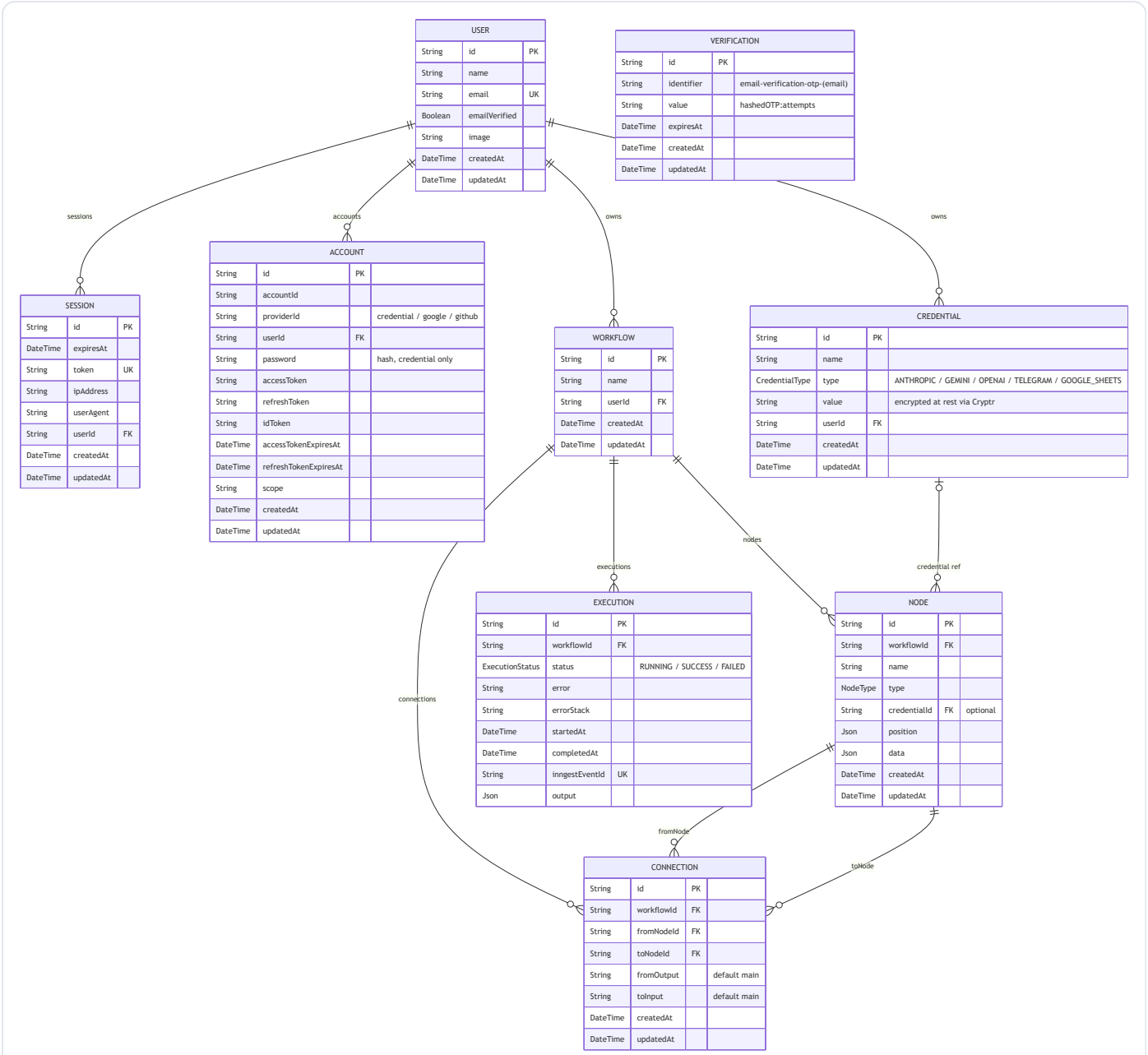


Figure 5 — Postgres ER diagram for Nodebase (Prisma schema)

7. Entity Reference

Table	Key fields	Relationships
User	name, email (unique), emailVerified, image	owns Workflow, Credential; has many Session, Account
Session	token (unique), expiresAt, ipAddress, userAgent	belongs to User (foreign key userId)
Account	accountId, providerId (credential / google / github), password (hash), accessToken, refreshToken, idToken, expiry timestamps, scope	belongs to User; OAuth tokens stored per provider
Verification	identifier (`email-verification-otp-<email>`), value (`hashedOTP:attempts`), expiresAt	standalone; drives email OTP lifecycle
Workflow	name	belongs to User; has many Node, Connection, Execution
Node	name, type, position (Json), data (Json), credentialId (optional)	belongs to Workflow; optional Credential; source/target of Connection
Connection	fromNodeId, toNodeId, fromOutput, toInput	belongs to Workflow; links two Node rows; drives topological sort
Credential	name, type, value (Cryptr-encrypted)	belongs to User; referenced by Node
Execution	status (RUNNING/SUCCESS/FAILED), error, errorStack, startedAt, completedAt, inngestEventId (unique), output (Json)	belongs to Workflow; linked 1:1 to an Inngest run

8. Node Catalog & Registration Model

Kind	Types	Notes
Triggers	MANUAL, CRON, GOOGLE_FORM, STRIPE, TELEGRAM	Webhook triggers take the workflow id as a query param and normalize the payload into `initialData` keys (`googleForm` / `stripe`) before dispatch.
AI actions	OPENAI, GEMINI, ANTHROPIC	Executors decrypt the linked credential (never expose to client), build the provider client, and return the text/structured answer into context.
Transports	HTTP_REQUEST, DISCORD, SLACK, TELEGRAM_SEND_MESSAGE, TELEGRAM_SEND_BUTTONS	HTTP executor uses ky + Handlebars templating; messaging executors publish to their own realtime channels.
Data	GOOGLE_SHEETS	Reads/writes sheet rows using the selected Google credential.

Adding a node type = 5 touchpoints

1. `NodeType` enum in `prisma/schema.prisma` + migration (`npx prisma migrate dev`).
2. Realtime channel in `src/inngest/channels/`.
3. Executor in `src/features/executions/components/<type>/executor.ts` + register in `executor-registry.ts`.
4. React Flow node in `<type>/node.tsx` + register in `src/config/node-components.ts`.
5. Config dialog `<type>/dialog.tsx` + node-selector entry.

Known copy-paste bug: `openai/executor.ts` (and gemini/anthropic) import/publish the wrong `anthropicChannel` — verify channel imports when editing AI executors.

9. Key Design Decisions & Notable Details

Security

- **Encrypted credentials at rest** (Cryptr); plaintext exists only inside the Inngest executor sandbox, never in API responses.
- **Procedural auth**: context is a stub; every real check happens in `protectedProcedure` / `premiumProcedure`, so routers can never silently bypass auth.
- **Polar premium gate**: `premiumProcedure` verifies the active subscription before premium mutations run.

Reliability & scale

- **Deterministic execution**: the graph is topologically sorted; cycles fail fast during `sendWorkflowExecution`.
- **Idempotency**: Upstash `SET NX EX` idempotency keys dedupe duplicated webhook deliveries.
- **Realtime topology**: Inngest realtime middleware carries per-node status to the browser without polling; executions remain queryable through Prisma history.
- **Bounded queues**: BullMQ email/cron workloads are in Redis; failed jobs are retained (last 1000) for debugging.

Deployment

- **AWS EKS migration in progress** — full plan, progress and active bugs live in `docs/deployment/STATUS.md`.
- Key ongoing issues: Docker build OOM (~2.7 GB) → mitigated with a global Prisma CLI in the image; outstanding `dotenv` runtime concern; `next.config.ts` sets `allowedDevOrigins: ["*"]` for ngrok.
- Vercel path builds via `npm run vercel-build` (`prisma generate && next build`) with the `rhel-openssl-3.0.x` Postgres binaryTarget.

Observations / possible improvements

- **`createTRPCContext` is a stub** — migrating the userId resolution into the context would centralize auth and remove the caveat.
- **`workflows.update` full graph replacement** is simple but loses node/connection ids between saves — a diff-based sync would reduce churn and aid undo.
- Executors publish channel status but the AI executors share a channel-naming bug (**anthropic channel imported in openai/gemini executors**).
- Execution retries (3) may re-run side-effectful HTTP/messaging nodes — consider dagger-free idempotent action guards for exactly-once semantics.